COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S3
DATE PERFORMED:	August 4, 2026	**DATE SUBMITTED:**	August 11, 2026
NAME:	Carlo Mari B. Asajar	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 5.1 QUEUES

1. PROCEDURE OUTPUT

ILO A: Create queue using C++ STL

Definition from the CPP reference documentation for the queue STL template. queues are a type of container adaptor, specifically designed to operate in a FIFO context (first-in first-out), where elements are inserted into one end of the container and extracted from the other. Queues are implemented as container adaptors, which are classes that use an encapsulated object of a specific container class as its underlying container, providing a specific set of member functions to access its elements. Elements are pushed into the "back" of the specific container and popped from its "front".

The underlying container may be one of the standard container class templates or some other specifically designed container class. This underlying container shall support at least the following operations:

- empty
- size
- front
- back
- push_back
- pop_front

The standard container classes deque and list fulfill these requirements. By default, if no container class is specified for a particular queue class instantiation, the standard container deque is used.

Function	Description
Constructor	Creates and initializes an empty queue object
Empty()	Checks whether the queue contains any elements. Returns true if empty, otherwise false.
Size()	Returns the number of elements currently stored in the queue.

Front()	Returns a reference to the element at the front of the queue (the next element to be removed).
Back()	Returns a reference to the element at the rear of the queue (the most recently added element).
Push()	Inserts a new element at the rear of the queue
Emplace()	Constructs a new element directly at the rear of the queue without creating a temporary object.
Pop()	Removes the element at the front of the queue.
Swap()	Exchanges the contents of one queue with another queue.

Task: Create C++ code using Queue STL that will pass an array of students' names to the queue. Make observations on the operations performed and show screenshots of code and output console per operation.

My Code:

ILOA > main_ILOA.cpp > main()

```cpp
#include <iostream>
#include <queue> // call the queue from the STL

void displayAll(std::queue<char> copyQueue);
void isEmpty(std::queue<char> emptyQueue);

int main() {
    // Create an object
    std::queue<char> myQ;

    // Testing the empty() function to check if the queue is empty
    std::cout << "\nIs the queue empty? " << std::endl;
    isEmpty(myQ);

    // Insert five elements into the queue
    myQ.push('C');
    myQ.push('A');
    myQ.push('R');
    myQ.push('L');
    myQ.push('O');

```

```
22      // Display all elements in the queue
23      std::cout << "\nElements in the queue: ";
24      displayAll(myQ);
25
26      // Testing the empty() function again after inserting elements
27      std::cout << "\nIs the queue empty? " << std::endl;
28      isEmpty(myQ);
29
30      // size() -> returns the total number of elements in the queue
31      std::cout << "\nSize of the queue: " << myQ.size() << "." << std::endl;
32
33      // front() -> returns the first element in the queue
34      std::cout << "\nFront of the queue: " << myQ.front() << std::endl;
35
36      // back() -> returns the last element in the queue
37      std::cout << "Back of the queue: " << myQ.back() << std::endl;
38
39      // push() -> inserts an element at the back of the queue
40      myQ.push('M');
41      std::cout << "\nAfter pushing 'M': ";
42      displayAll(myQ);
43
44      // pop() -> removes the first element from the queue
45      std::cout << "\nElement to be popped: " << myQ.front() << std::endl;
46      myQ.pop();
47
48      // Display the queue after removing the first element
49      std::cout << "Queue after popping: ";
```

```
50      displayAll(myQ);
51
52      return 0;
53  }
54
55  void displayAll(std::queue<char> copyQueue) {
56      // Creates a copy of the queue that was passed
57      std::queue<char> c = copyQueue;
58
59      while (!c.empty()) {
60          std::cout << c.front();
61          c.pop();
62      }
63
```

```
64      std::cout << std::endl;
65  }
66
67  void isEmpty(std::queue<char> emptyQueue) {
68      // empty() -> checks if the queue contains any elements
69      if (emptyQueue.empty()) {
70          std::cout << "Queue is empty." << std::endl;
71      } else {
72          std::cout << "Queue is not empty." << std::endl;
73      }
74  }
```

My Output:

```
Is the queue empty?
Queue is empty.

Elements in the queue: CARLO

Is the queue empty?
Queue is not empty.

Size of the queue: 5.

Front of the queue: C
Back of the queue: O

After pushing 'M': CARLOM

Element to be popped: C
Queue after popping: ARLOM
```

My Analysis:

The code demonstrates the basic operations of a C++ STL queue by creating ***std::queue<char> myQ*** and using it to store the characters ***C, A, R, L,*** and ***O***. The program first calls ***isEmpty(myQ)*** before adding anything, so the first output shows ***"Queue is empty."*** This happens because ***std::queue<char> myQ*** is newly created and has no elements yet. The five ***push()*** statements then add ***C, A, R, L,*** and ***O*** to the back of the queue, following the ***FIFO*** order. The ***displayAll(myQ)*** function is important because it receives a copy of the queue, creates another copy using ***std::queue<char> c = copyQueue***, and then uses ***c.front()*** and ***c.pop()*** inside the while loop to print every element. Because the copy is used, the original queue remains unchanged after displaying it.

The program then calls i***sEmpty(myQ)*** again, which prints ***"Queue is not empty."*** because the five elements are already stored. The ***myQ.size()*** statement prints ***5***, while ***myQ.front()*** prints ***C*** because it is the first element in the queue, and ***myQ.back()*** prints ***O*** because it is the last element added. After that, ***myQ.push('M')*** adds ***M*** to the back, so ***displayAll(myQ)*** prints ***CARLOM*** in the same order. Before removing an element, **myQ.front()** is printed again, showing *C* as the element that will be removed. The ***myQ.pop()*** statement then removes ***C*** from the front, following the ***FIFO*** rule, and the final ***displayAll(myQ)*** prints ***ARLOM***.

B.1. Linked List Implementation

For this section, the linked list implementation will focus on the given behavior of operations as they operate on the nodes and link them together. The nodes are defined the same way as previously, with two compartments: data and pointer to the next.

Operations:

- Inserting an item into a non-empty queue
- Inserting an item into an empty queue
- Deleting an item from a queue of more than one item
- Deleting an item from a queue with one item

For each item, you are provided an algorithm for the operations. You must implement C++ code for each operation. Then, show each operation and its output as shown through the console.

My Code: (queueLL_ILOB.h)

```
ILOB1 > C queueLL_ILOB.h > dequeue<T>(qNode<T>**, qNode<T>**)
  1   #ifndef QUEUELL_ILOB_H
  2   #define QUEUELL_ILOB_H
  3
```

```cpp
#include <iostream>

// Create a class for the node
template <typename T>
class qNode{
    public:
        T data;
        qNode<T>* next;
};

// Create the new node
template <typename T>
qNode<T>* new_node(T newData){
    qNode<T>* newNode = new qNode<T>;
    newNode->data = newData;
    newNode->next = nullptr;

    return newNode;
}

// Enqueue operation
template <typename T>
void enqueue(T newData, qNode<T>** frontPtr, qNode<T>** backPtr){
    // Create the new node
    qNode<T>* newNode = new_node(newData);

    // Inserting an item into an empty queue
    if((*frontPtr) == nullptr && (*backPtr) == nullptr){
        (*frontPtr) = newNode;
        (*backPtr) = newNode;

        return;
    }

    // Inserting an item into a non-empty queue
    (*backPtr)->next = newNode;
    (*backPtr) = newNode;
}

```

```cpp
// Dequeue operation
template <typename T>
void dequeue(qNode<T>** frontPtr, qNode<T>** backPtr){
    // Check if the queue is empty
    if((*frontPtr) == nullptr && (*backPtr) == nullptr){
        std::cout << "Queue is empty." << std::endl << std::endl;

        return;
    }

    // Create a temporary node
    qNode<T>* deleteNode = nullptr;
    deleteNode = (*frontPtr);

    // Deleting an item from a queue with one item
    if((*frontPtr)->next == nullptr){
        (*frontPtr) = nullptr;
        (*backPtr) = nullptr;

        delete deleteNode;

        return;
    }

    // Deleting an item from a queue with more than one item
    (*frontPtr) = (*frontPtr)->next;

    deleteNode->next = nullptr;
    delete deleteNode;
}

// Check the front
template <typename T>
void checkFront(qNode<T>* frontPtr){
    // Check if the queue is empty
    if(frontPtr == nullptr){
        std::cout << "Queue is empty." << std::endl << std::endl;

        return;
    }

```

```
84      // Display the front element
85      std::cout << "Current Front: " << frontPtr->data << std::endl;
86  }
87
88  // Check the back
89  template <typename T>
90  void checkBack(qNode<T>* backPtr){
91      // Check if the queue is empty
92      if(backPtr == nullptr){
93          std::cout << "Queue is empty." << std::endl << std::endl;
94
```

```
95          return;
96      }
97
98      // Display the back element
99      std::cout << "Current Back: " << backPtr->data << std::endl;
100 }
101
102 // Display all elements
103 template <typename T>
104 void displayAll(qNode<T>* frontPtr){
105     // Check if the queue is empty
106     if(frontPtr == nullptr){
107         std::cout << "Queue is empty." << std::endl << std::endl;
108
109         return;
110     }
111
112     // Display all elements in the queue
113     std::cout << "Current Queue: ";
114
115     while(frontPtr != nullptr){
116         std::cout << frontPtr->data << " ";
117         frontPtr = frontPtr->next;
118     }
119
120     std::cout << std::endl;
121 }
122
123 #endif
```

My Code: (main_ILOB1.cpp)

ILOB1 > main_ILOB1.cpp > main()

```cpp
#include <iostream>
#include "queueLL_ILOB.h"

int main(){

    qNode<char>* front = nullptr;
    qNode<char>* back = nullptr;

    // Inserting an item into an empty queue
    enqueue('C', &front, &back);
    std::cout << front->data << " " << back->data << std::endl;

    // Inserting an item into a non-empty queue
    enqueue('A', &front, &back);
    std::cout << front->data << " " << back->data << std::endl;
    enqueue('R', &front, &back);
    std::cout << front->data << " " << back->data << std::endl;
    enqueue('L', &front, &back);
    std::cout << front->data << " " << back->data << std::endl;
    enqueue('O', &front, &back);
    std::cout << front->data << " " << back->data << std::endl << std::endl;

    // Display all elements in the queue
    displayAll(front);

    // Check the front of the queue
    checkFront(front);

    // Check the back of the queue
    checkBack(back);

    std::cout << std::endl;
```

```cpp

    // Deleting an item from a queue of more than one item
    dequeue(&front, &back);
    std::cout << front->data << " " << back->data << std::endl;
    dequeue(&front, &back);
    std::cout << front->data << " " << back->data << std::endl;
    dequeue(&front, &back);
    std::cout << front->data << " " << back->data << std::endl << std::endl;

    // Display the queue
```

```
43      displayAll(front);
44
45      // Delete an item until only one item remains
46      dequeue(&front, &back);
47
48      // Deleting an item from a queue with one item
49      dequeue(&front, &back);
50
51      // Display the queue after deleting the last item
52      displayAll(front);
53
54      return 0;
55  }
```

My Output:

```
C C
C A
C R
C L
C O

Current Queue: C A R L O
Current Front: C
Current Back: O

A O
R O
L O

Current Queue: L O
Queue is empty.
```

My Analysis:

The code shows how a queue using a linked list works by using the ***qNode<T>*** class to store the data and the ***next*** pointer that connects each node to the next one. The ***front*** and ***back*** pointers are very important because they keep track of the first and last elements of the queue. The ***new_node()*** function creates a new node and sets its ***data*** and ***next*** values, while the ***enqueue()*** function adds new elements at the back of the queue. The code also handles both an empty queue and a non-empty queue by checking if ***(*frontPtr) == nullptr && (*backPtr) == nullptr*** before inserting the first element. In the main file, ***C, A, R, L,*** and ***O*** are added using ***enqueue()***, showing that the elements follow the FIFO order.

The ***displayAll(front)*** function then goes through the linked nodes using ***frontPtr = frontPtr->next*** to show all the elements. The ***checkFront()*** and ***checkBack()*** functions are also used to show the first and last elements, which helps in checking if the queue is working correctly. The ***dequeue()*** function is important because it removes the element from the front and also uses ***delete*** to free the removed node from memory. It also has separate conditions for a queue with more than one item and a queue with only one item, making the deletion process more complete.

B.2. Array-Based Implementation

In C++, we could implement an array-based queue as a class. To conserve space, we'll implement it as a "circular queue", an array in which the last position is logically connected back to the first position to make a circle. This is sometimes also called a "ring buffer"

Data members

- q_array - Queue array pointer. A pointer to the data type of the items stored in the queue; points to the first element of a dynamically-allocated array.
- q_capacity - Queue capacity. The number of elements in the queue array.
- q_size - Queue size. The number of items currently stored in the queue.
- q_front - Queue front. The subscript of the front (or head) item in the queue.
- q_back - Queue back. The subscript of the back (or rear or tail) item in the queue.

Member Functions

The example functions described here correspond to the interface of the queue class in the C++ standard library.

Operation	Description
Queue() (Constructor)	Constructs an empty queue.

Empty()	Tests whether the queue is empty. Returns true if the queue contains no elements; otherwise returns false.
Size()	Returns the number of elements currently stored in the queue.
Clear()	Resets the queue size to 0 and restores q_front and q_back to their initial values. Does not deallocate dynamic memory or change the queue capacity
Front()	Returns the element at the front of the queue (the next element to be removed).
Back()	Returns the element at the back of the queue (the most recently added element).
Enqueue()	Inserts a new element at the back of the queue.
Dequeue()	Removes the element at the front of the queue.
Copy Constructor	Creates a new queue as a copy of an existing queue. Unlike vectors or array-based stacks, queue elements may wrap around the array, so the entire queue array must be copied.
Copy Assignment Operator	Replaces the contents of one queue with a copy of another queue. Since queue elements may not occupy consecutive positions, the entire queue array must be copied.
Destructor	Deletes the dynamically allocated queue array and releases its memory.

Member Functions (modified from STL) [bold texts are added modifications]

Your Tasks:

- Create a queue class that will contain the array.
- Operations and member functions described in this section must be in your developed code.
- Test the functions and show the output.

My Code: (queueArr_ILOB2.h)

ILOB2 > queueArr_ILOB2.h > QueueArr(size_t)

```h
#ifndef QUEUEARR_ILOB2_H
#define QUEUEARR_ILOB2_H

#include <cstddef>
#include <iostream>

template <typename T>
class QueueArr{

    private:

        T* q_Array;
        size_t q_capacity;
        size_t q_size;
        int q_front;
        int q_back;

    public:

        //Construct empty queue
        QueueArr(size_t capacity);

        //Destructor
        ~QueueArr();

        bool Empty();
        bool Full();

        size_t Size();

        void Clear();

        //Check the front
        T Front();

        //Check the back
        T Back();

        //Enqueue operation
        void Enqueue(T newData);

        //Dequeue operation
```

```
43          void Dequeue();
44
45          //Copy constructor
46          QueueArr(const QueueArr& other);
47
48          //Copy assignment operator
49          QueueArr& operator=(const QueueArr& other);
50  };
51
52  //definition of constructor
53  template <typename T>
54  QueueArr<T>::QueueArr(size_t q_capacity){
55
56      //capacity
57      this->q_capacity = q_capacity;
58
59      //size = 0
60      q_size = 0;
61
62      //front = 0
63      q_front = 0;
64
65      //back = -1
66      q_back = -1;
67
68      //update capacity
69      q_Array = new T[q_capacity];
70
71      std::cout << "Constructor was called." << std::endl;
72
73  }
74
```

```
75  //Destructor
76  template <typename T>
77  QueueArr<T>::~QueueArr(){
78
79      delete[] q_Array;
80
81      std::cout << "Destructor was called." << std::endl;
82  }
83
84  //isempty
85  template <typename T>
86  bool QueueArr<T>::Empty(){
```

```
87
88      return (q_size == 0);
89
90  }
91
92  //isfull
93  template <typename T>
94  bool QueueArr<T>::Full(){
95
96      return (q_size == q_capacity);
97  }
98
99  //size
100 template <typename T>
101 size_t QueueArr<T>::Size(){
102
103     return q_size;
104 }
105
106 //clear
107 template <typename T>
108 void QueueArr<T>::Clear(){
109
110     //queue size = 0
111     q_size = 0;
112
113     //queue front and queue back to 0 and -1
114     q_front = 0;
115     q_back = -1;
116
117     return;
118 }
119
```

```
120 //front
121 template <typename T>
122 T QueueArr<T>::Front(){
123
124     //check if queue is empty
125     //if not empty, show the front
126     if(Empty()){
127         std::cout << "Queue is empty" << std::endl;
128
129         return T{};
130     }
131
132     else{
```

```
133             return q_Array[q_front];
134         }
135     }
136
137     //back
138     template <typename T>
139     T QueueArr<T>::Back(){
140
141         //check if queue is empty
142         //if not empty, show the back
143         if(Empty()){
144             std::cout << "Queue is empty" << std::endl;
145
146             return T{};
147         }
148
149         else{
150             return q_Array[q_back];
151         }
152     }
153
154     //enqueue
155     template <typename T>
156     void QueueArr<T>::Enqueue(T newData){
157
158         //check if queue is full
159         if(Full()){
160             std::cout << "Queue is full" << std::endl;
161             return;
162         }
163         else{
164
```

```
165             //compute back
166             q_back = (q_back + 1) % q_capacity;
167
168             //add new data to the back
169             q_Array[q_back] = newData;
170
171             //increment size
172             q_size++;
173         }
174     }
175
176     //dequeue
```

```cpp
template <typename T>
void QueueArr<T>::Dequeue(){

    //check if queue is empty
    if(Empty()){
        std::cout << "Queue is empty" << std::endl;
        return;
    }
    else{

        //compute front
        q_front = (q_front + 1) % q_capacity;

        //decrement size
        q_size--;
    }
}

//copy constructor
template <typename T>
QueueArr<T>::QueueArr(const QueueArr& other){

    //copy the queue capacity
    q_capacity = other.q_capacity;

    //copy the queue size
    q_size = other.q_size;

    //copy the queue front
    q_front = other.q_front;
```

```cpp

    //copy the queue back
    q_back = other.q_back;

    //create a new array
    q_Array = new T[q_capacity];

    //copy the elements of the queue array
    for(size_t i = 0; i < q_capacity; i++){
        q_Array[i] = other.q_Array[i];
    }
}

```

```cpp
//copy assignment operator
template <typename T>
QueueArr<T>& QueueArr<T>::operator=(const QueueArr& other){

    //check if the object is not assigned to itself
    if(this != &other){

        //delete the existing array
        delete[] q_Array;

        //copy the queue capacity
        q_capacity = other.q_capacity;

        //copy the queue size
        q_size = other.q_size;

        //copy the queue front
        q_front = other.q_front;

        //copy the queue back
        q_back = other.q_back;

        //create a new array
        q_Array = new T[q_capacity];

        //copy the elements of the queue array
        for(size_t i = 0; i < q_capacity; i++){
            q_Array[i] = other.q_Array[i];
        }
    }

    return *this;
}

#endif
```

My Code: (main_ILOB2.cpp)

ILOB2 > main_ILOB2.cpp > main()

```cpp
#include <iostream>
#include "queueArr_ILOB2.h"

int main(){

    //create an object
    QueueArr<char> q(6);

    //Test the empty() function
    std::cout << "Is queue empty? "
              << (q.Empty() ? "Yes" : "No") << std::endl;

    //Insert elements into the queue
    q.Enqueue('C');
    q.Enqueue('A');
    q.Enqueue('R');
    q.Enqueue('L');
    q.Enqueue('O');

    //Check the front and back of the queue
    std::cout << "Front: " << q.Front() << std::endl;
    std::cout << "Back: " << q.Back() << std::endl;

    //Check the size of the queue
    std::cout << "Size: " << q.Size() << std::endl;

    //Remove the front element from the queue
    q.Dequeue();

    std::cout << "Front after dequeue: "
              << q.Front() << std::endl;

    //Insert an element after dequeue
    q.Enqueue('M');

    std::cout << "After enqueueing M:" << std::endl;
    std::cout << "Front: " << q.Front() << std::endl;
    std::cout << "Back: " << q.Back() << std::endl;
    std::cout << "Size: " << q.Size() << std::endl;

    //Clear the queue
    q.Clear();
```

```
43
44      std::cout << "Is queue empty after clear? "
45                << (q.Empty() ? "Yes" : "No") << std::endl;
46
47      //Insert elements into the queue
48      q.Enqueue('C');
49      q.Enqueue('A');
50      q.Enqueue('R');
51
52      //Test the copy constructor
53      QueueArr<char> qCopy(q);
54
55      std::cout << "Copy constructor front: "
56                << qCopy.Front() << std::endl;
57      std::cout << "Copy constructor back: "
58                << qCopy.Back() << std::endl;
59
60      //Test the copy assignment operator
61      QueueArr<char> qAssign(6);
62
63      qAssign = q;
64
65      std::cout << "Copy assignment front: "
66                << qAssign.Front() << std::endl;
67      std::cout << "Copy assignment back: "
68                << qAssign.Back() << std::endl;
69
70      return 0;
71
72  }
```

My Output:

```
Constructor was called.
Is queue empty? Yes
Front: C
Back: O
Size: 5
Front after dequeue: A
After enqueueing M:
Front: A
Back: M
Size: 5
Is queue empty after clear? Yes
Copy constructor front: C
Copy constructor back: R
Constructor was called.
Copy assignment front: C
Copy assignment back: R
Destructor was called.
Destructor was called.
Destructor was called.
```

My Analysis:

The code shows how an array-based circular queue works by using the ***QueueArr<T>*** class and the important data members ***q_Array***, ***q_capacity***, ***q_size***, ***q_front***, and ***q_back*** to manage the elements. The constructor ***QueueArr(size_t capacity)*** creates the dynamic array using ***new T[q_capacity]*** and sets the starting values of the queue, while the destructor ***~QueueArr()*** uses ***delete[] q_Array*** to release the memory after the program finishes. The ***Empty()*** and ***Full()*** functions check if the queue has no elements or has reached its capacity, while ***Size()*** returns the current number of elements. The ***Enqueue()*** function is important because it adds new elements at the back, and the expression ***q_back = (q_back + 1) % q_capacity*** allows the queue to move back to the beginning of the array, which is what makes it circular.

The ***Dequeue()*** function removes the front element by updating ***q_front*** and decreasing ***q_size***, while ***Front()*** and ***Back()*** are used to check the first and last elements. The ***Clear()*** function resets the queue by setting ***q_size*** to ***0***, ***q_front*** to ***0***, and ***q_back*** to ***-1*** without deleting the allocated array. The main file tests these operations by adding ***C, A, R, L,*** and ***O***, checking the front, back, and size, removing an element, and then adding ***M*** again to show that the queue can continue working after a dequeue. The copy constructor and copy assignment operator are also tested using ***qCopy*** and ***qAssign***, showing that another queue can receive the same contents without directly sharing the same dynamic array.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO C: Solve problems using queue implementation

Problem Title: Shared Printer Simulation using Queues
Problem Definition: In this activity, we'll simulate a queue for a shared printer in an office. In any corporate office, usually, the printer is shared across the whole floor in the printer room. All the computers in this room are connected to the same printer. But a printer can do only one printing job at any point in time, and it also takes some time to complete any job. In the meantime, some other user can send another print request. In such a case, a printer needs to store all the pending jobs somewhere so that it can take them up once its current task is done.

Perform the following steps to solve the activity. Make sure that you include a screenshot of the source code for each. From the get-go: You must choose whether you are making a linked list or array implementation. You may NOT use the STL.

1. Create a class called Job (comprising an ID for the job, the name of the user who submitted it, and the number of pages).
2. Create a class called Printer. This will provide an interface to add new jobs and process all the jobs added so far.
3. To implement the printer class, it will need to store all the pending jobs. We'll implement a very basic strategy – first come, first served. Whoever submits the job first will be the first to get the job done.
4. Finally, simulate a scenario where multiple people are adding jobs to the printer, and the printer is processing them one by one.
5. Defend your choice of internal representation: Why did you use arrays or linked lists?

Output Analysis:

- Provide the output after performing each task above.
- Include your analysis: focus on why you think the output is the way it is.

My Code: (job_ILOC.h)

ILOC > C job_ILOC.h > Job

```
1   #ifndef JOB_ILOC_H
2   #define JOB_ILOC_H
3
4   #include <iostream>
5   #include <iomanip>
6   #include <string>
7
8   class Job {
9   public:
10      int id;
11      std::string user;
12      int pages;
13
14      // Constructor
15      Job(int i = 0, std::string u = "", int p = 0) {
16          id = i;
17          user = u;
18          pages = p;
19      }
20
21      // Display job information
22      void display() {
23          std::cout << std::left
24                    << std::setw(10) << id
25                    << std::setw(20) << user
26                    << std::setw(10) << pages
27                    << std::endl;
28      }
29  };
30
31  #endif
```

My Code: (printer_ILOC.h)

ILOC > C printer_ILOC.h > ...

```h
#ifndef PRINTER_ILOC_H
#define PRINTER_ILOC_H

#include <iostream>
#include <iomanip>
#include "job_ILOC.h"

class Printer {
private:
    Job* q_array;
    int q_capacity;
    int q_size;
    int q_front;
    int q_back;

public:
    // Constructor
    Printer(int capacity = 10) {
        q_capacity = capacity;
        q_array = new Job[q_capacity];
        q_size = 0;
        q_front = 0;
        q_back = -1;
    }

    // Destructor
    ~Printer() {
        delete[] q_array;
    }

```

```h
    // Add a new print job to the queue
    void addJob(Job j) {
        if (q_size == q_capacity) {
            std::cout << "Queue is full. Job cannot be added.\n";
            return;
        }

        q_back = (q_back + 1) % q_capacity;
        q_array[q_back] = j;
        q_size++;
```

```
41
42          std::cout << "Job added: ";
43          j.display();
44      }
45
46      // Process the next print job
47      void processJob() {
48          if (q_size == 0) {
49              return;
50          }
51
52          q_array[q_front].display();
53
54          q_front = (q_front + 1) % q_capacity;
55          q_size--;
56      }
57
58      // Display all pending jobs
59      void displayQueue() {
60          if (q_size == 0) {
61              std::cout << "Queue is empty.\n";
62              return;
63          }
64
65          std::cout << std::left
66                    << std::setw(10) << "ID"
```

```
67                    << std::setw(20) << "User"
68                    << std::setw(10) << "Pages"
69                    << std::endl;
70
71          std::cout << "-----------------------------------------------\n";
72
73          for (int i = 0; i < q_size; i++) {
74              int index = (q_front + i) % q_capacity;
75              q_array[index].display();
76          }
77
78          std::cout << "-----------------------------------------------\n";
79      }
80  };
81
82  #endif
```

My Code: (main_ILOC.cpp)

ILOC > main_ILOC.cpp > main()

```cpp
#include <iostream>
#include <iomanip>
#include "job_ILOC.h"
#include "printer_ILOC.h"

int main() {
    Printer printer(5);

    std::cout << "            SHARED PRINTER SIMULATION\n";
    std::cout << "=============================================\n";

    // Add jobs
    std::cout << "\n================= Adding Jobs =================\n";
    std::cout << "-----------------------------------------------\n";

    printer.addJob(Job(1, "Carlo A.", 12));
    printer.addJob(Job(2, "Ralf A.", 7));
    printer.addJob(Job(3, "Michael B.", 15));
    printer.addJob(Job(4, "Troy B.", 4));

    // Display current printing queue
    std::cout << "\n=========== Current Printing Queue ============\n";
    std::cout << "-----------------------------------------------\n";

    printer.displayQueue();

    // Process jobs
    std::cout << "\n================ Processing Jobs ===============\n";
    std::cout << "-----------------------------------------------\n";
    std::cout << std::left
              << std::setw(10) << "ID"
              << std::setw(20) << "User"
              << std::setw(10) << "Pages"
              << std::endl;

    std::cout << "-----------------------------------------------\n";

    printer.processJob(); // Printing for Carlo A.
    printer.processJob(); // Printing for Ralf A.

    std::cout << "-----------------------------------------------\n";
```

```
40
41      std::cout << "-----------------------------------------------\n";
42
43      // Display queue after processing
44      std::cout << "\n============ Queue After Processing ============\n";
45      std::cout << "-----------------------------------------------\n";
46
47      printer.displayQueue();
48
49      // Add more jobs
50      std::cout << "\n============== Adding More Jobs ===============\n";
51      std::cout << "-----------------------------------------------\n";
52
53      printer.addJob(Job(5, "Emmanuel B.", 10));
54      printer.addJob(Job(6, "Jeffordniel C.", 6));
55      printer.addJob(Job(7, "Lorenzo D.", 9));
56
57      // Display final queue
58      std::cout << "\n============ Final Printing Queue =============\n";
59      std::cout << "-----------------------------------------------\n";
60
61      printer.displayQueue();
62
63      // Process remaining jobs
64      std::cout << "\n========== Processing Remaining Jobs ==========\n";
65      std::cout << "-----------------------------------------------\n";
66
67      std::cout << std::left
68                << std::setw(10) << "ID"
69                << std::setw(20) << "User"
70                << std::setw(10) << "Pages"
```

```
71                << std::endl;
72
73      std::cout << "-----------------------------------------------\n";
74
75      printer.processJob(); // Printing for Michael B.
76      printer.processJob(); // Printing for Troy B.
77      printer.processJob(); // Printing for Emmanuel B.
78      printer.processJob(); // Printing for Jeffordniel C.
79      printer.processJob(); // Printing for Lorenzo D.
80
81      std::cout << "-----------------------------------------------\n";
82
```

```
83      // Display queue after all jobs are processed
84      std::cout << "\n============= Queue After All Jobs ==============\n";
85      std::cout << "-------------------------------------------------\n";
86
87      printer.displayQueue();
88
89      return 0;
90  }
```

My Output:

Part 1

```
            SHARED PRINTER SIMULATION
================================================

================ Adding Jobs =================
------------------------------------------------
Job added: 1        Carlo A.          12
Job added: 2        Ralf A.           7
Job added: 3        Michael B.        15
Job added: 4        Troy B.           4

=========== Current Printing Queue ============
------------------------------------------------
ID      User              Pages
------------------------------------------------
1       Carlo A.          12
2       Ralf A.           7
3       Michael B.        15
4       Troy B.           4
------------------------------------------------

=============== Processing Jobs ===============
------------------------------------------------
ID      User              Pages
------------------------------------------------
1       Carlo A.          12
2       Ralf A.           7
------------------------------------------------

=========== Queue After Processing ============
------------------------------------------------
ID      User              Pages
------------------------------------------------
3       Michael B.        15
4       Troy B.           4
------------------------------------------------
```

Part 2

```
============= Adding More Jobs ===============
------------------------------------------------
Job added: 5        Emmanuel B.       10
Job added: 6        Jeffordniel C.    6
Job added: 7        Lorenzo D.        9

============ Final Printing Queue =============
------------------------------------------------
ID      User              Pages
------------------------------------------------
3       Michael B.        15
4       Troy B.           4
5       Emmanuel B.       10
6       Jeffordniel C.    6
7       Lorenzo D.        9
------------------------------------------------

========== Processing Remaining Jobs ==========
------------------------------------------------
ID      User              Pages
------------------------------------------------
3       Michael B.        15
4       Troy B.           4
5       Emmanuel B.       10
6       Jeffordniel C.    6
7       Lorenzo D.        9
------------------------------------------------

============ Queue After All Jobs =============
------------------------------------------------
Queue is empty.
PS C:\Users\carlo\Downloads\ASAJAR_CPE 010_Hands-
 Activity 5.1_Queue>
```

My Analysis:

The code shows how a shared printer queue can be simulated using an array-based circular queue without using the STL. The **Job** class stores the important information for every printing request, which are the ***id***, ***user***, and ***pages***, while the ***Printer*** class manages the pending jobs using ***q_array***, ***q_capacity***, ***q_size***, ***q_front***, and ***q_back***. The ***addJob()*** function is important because it places each new job at the back of the queue using ***q_back = (q_back + 1) % q_capacity***, allowing the array to work as a circular queue. The ***processJob()*** function gets the job at ***q_front***, displays its information, moves the front position forward, and decreases ***q_size***, which shows the ***FIFO*** or First Come, First Served behavior. An array-based implementation was chosen because it is simple to manage and works well for a printer with a known queue capacity. The circular queue also allows the empty spaces from completed jobs to be reused instead of creating new nodes, which makes the storage more organized. In the printed output, the first section shows the jobs being added, where ***Job 1*** for ***Carlo A.*** has ***12 pages***, ***Job 2*** for ***Ralf A***. has ***7 pages***, ***Job 3*** for ***Michael B***. has ***15 pages***, and ***Job 4*** for ***Troy B***. has ***4 pages***. The ***Current Printing Queue*** then shows the same jobs in the same order, proving that they were stored correctly.

During the ***Processing Jobs*** section, ***Carlo A***. and ***Ralf A***. appear first in the output because they were the first two jobs added. After processing them, the ***Queue After Processing*** output shows ***Michael B***. and ***Troy B***. as the remaining jobs. More jobs are then added, including ***Emmanuel B.*** with ***10 pages***, ***Jeffordniel C.*** with ***6 pages***, and ***Lorenzo D.*** with ***9 pages***, and the ***Final Printing Queue*** shows these jobs after the two remaining earlier jobs. The ***Processing Remaining Jobs*** section then displays ***Michael, Troy, Emmanuel, Jeffordniel,*** and ***Lorenzo*** in that exact order, which confirms that the queue follows FIFO properly. The ***displayQueue()*** function is also important because the expression ***(q_front + i) % q_capacity*** allows the jobs to be displayed correctly even when the positions wrap around the array. The use of ***std::setw()*** in ***Job::display()*** also makes the printed output more organized by keeping the ***ID, user,*** and ***pages*** in separate aligned columns. Finally, the ***Queue After All Jobs*** output shows that the queue is empty, proving that every pending job was processed successfully.

Conclusion:

The activity helped show how queues work in different ways in C++ while following the FIFO rule, where the first element added is also the first one removed. From the procedures, the use of the STL queue, linked list, and array-based circular queue helped make the differences between each implementation more understandable. The STL part helped in learning basic operations like push(), pop(), front(), back(), and size(), while the linked list part showed how nodes, pointers, front, and back are connected when adding and removing elements. The array-based part helped in understanding how q_front, q_back, and the % operator allow the queue to reuse spaces in a circular way. The supplementary activity about the shared printer made the lesson more practical because the printing jobs were processed one by one based on the order they were submitted.

Some difficulties were faced while understanding pointers, dynamic memory, and the movement of the front and back positions, especially when the circular queue returned to the beginning of the array. The output also needed to be checked carefully to make sure that the jobs and elements were still following the correct FIFO order. Overall, the activity was done fairly well because the different queue implementations were successfully understood and the printed outputs showed the expected results. There is still some area for improvement, especially in understanding pointers, memory allocation, circular indexing, and the copy operations because these parts can still be confusing sometimes. More practice with creating queue programs without following an example would help make the process more familiar and improve confidence in writing and checking the code.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **ChatGPT, Grammarly**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **formatting, explanation of concepts, grammar correction**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **The AI was used to explain how the code works by giving a detailed explanation on the different parts of the code. Moreover, the AI was used to assist in improving the grammar of the code analysis, allowing the explanation to be written more clearly and fluently in English.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.